MIGUEL VELO ARRILLAGA

COHABIA

2ºDesarrollo de Aplicaciones Multiplataforma

Trabajo Fin de Ciclo

1. INTRODUCCIÓN

1.1. Descripción

Se trata de una aplicación móvil dirigida a aquellas personas que necesitan centralizar toda la información sobre su casa en un mismo lugar. La app permitirá a los usuarios registrarse, crear un grupo de convivencia o unirse a uno ya existente y, desde ahí, gestionar aspectos esenciales como:

- Un calendario compartido
- Planificador de eventos
- Lista de la compra compartida
- Seguimiento y división de gastos
- Control de electrodomésticos
- Tablón de anuncios y notificaciones

Desarrollada en Kotlin con Jetpack Compose, utilizando Firebase como backend (Firestore, Authentication, Storage y Firebase Cloud Messaging).

La arquitectura de la aplicación está pensada para ser escalable y modular, facilitando futuras ampliaciones o mantenimiento.

El público objetivo de esta aplicación es el sector de la población que comparte casa de cualquier forma, interesado en la tecnología y en centralizar en un dispositivo sus datos.

1.2. Motivación

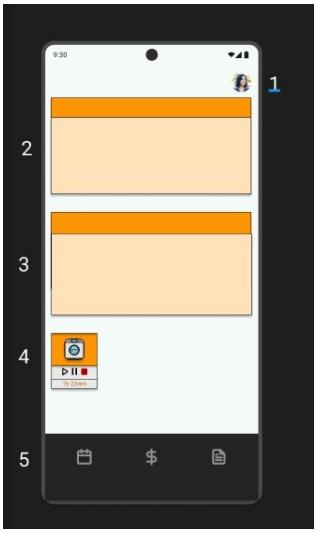
Esta idea de proyecto viene de mi reciente experiencia al independizarme. Esto me ha hecho ver que una aplicación así podría resultarme muy útil para gestionar en común más fácilmente todo lo que una casa conlleva. A lo largo de este proceso, he notado que, aunque existen aplicaciones que cubren algunas de las funcionalidades descritas, ninguna tiene todas de la manera que necesito ni que integre todas ellas en una sola plataforma. Además, considero que esta necesidad no es exclusiva de mi caso, sino que puede ser relevante para muchas más personas que comparten vivienda, ya sea con su familia, pareja, amigos o compañeros de piso.

1.3. Diseño

El diseño previo de la aplicación ha sido realizado con Figma, herramienta que me ha permitido modelar toda la interfaz.

Vistas:

(Las imágenes son esquemáticas y no representan el diseño final)



Pantalla inicio:

1:Foto de perfil, menú desplegable de cuenta, ajustes , etc 2:”Vistazo rápido”: últimos eventos y notificaciones.
3: Acceso a otras partes de la aplicación
4: Electrodomésticos activos
5: Menu navegación

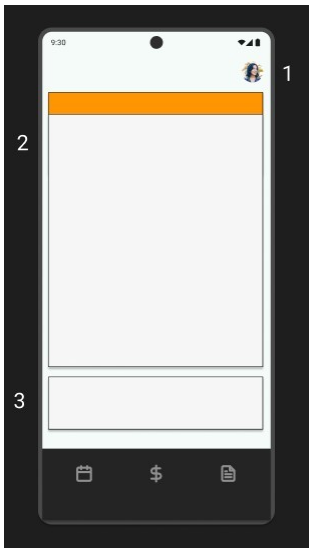
Pantalla con lista de elementos:

Esta pantalla representa a todas aquellas que posean una lista de elementos como puede ser la de ajustes, notas o notificaciones.

La barra naranja sería el título y los cuadrados blancos los elementos.

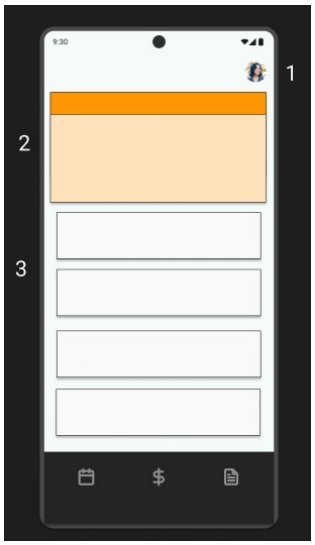
Pantalla de Login:

1: Logo de la app



Pantalla de calendario y eventos:

1:Foto perfil
2:Calendario donde se muestran eventos
3:Evento de hoy en mayor detalle



Pantalla finanzas:

1:Foto perfil
2: Vistazo rápido de finanzas
3: Acceso a otros menús de finanzas

Pantalla creación de electrodomésticos

1:Foto perfil
2:Iconos a elegir
3:Formulario

2 GESTIÓN Y ORGANIZACIÓN DEL PROYECTO

2.1. **Metodología de desarrollo**

En este proyecto se ha usado la metodología Desarrollo Rápido de Aplicaciones (RAD) ,basada en implementaciones sucesivas. Este método es ideal para este proyecto al estar estructurado en módulos. De esta manera, además, es posible tener un desarrollo más fácilmente adaptable al tiempo establecido.

El proceso por cada módulo desarrollado ha sido:

1. Estudio de la funcionalidad: se evalúa qué requerimientos tiene y cómo se va a llevar a cabo
2. Diseño: Utilización de herramientas como Figma para establecer cúal va a ser el diseño a seguir a la hora de programar
3. Desarrollo e implementación del código: branch dedicada a la funcionalidad en la que se desarrolla la nueva funcionalidad
4. Pruebas: verificación del correcto funcionamiento, pruebas manuales y procesos de debugging.
5 Fusión a rama principal: una vez verificado todo se une y se pasa a la siguiente funcionalidad

A pesar de llevar a la práctica esta metodología algunos de estos pasos siempre están sujetos a revisión. Por ejemplo, a pesar de haber fusionado ya la rama es posible que surjan cambios de diseño, errores no detectados o cambios de estructura en el código.

2.2 **Planificación**

2.2.1 Tabla de planificación inicial

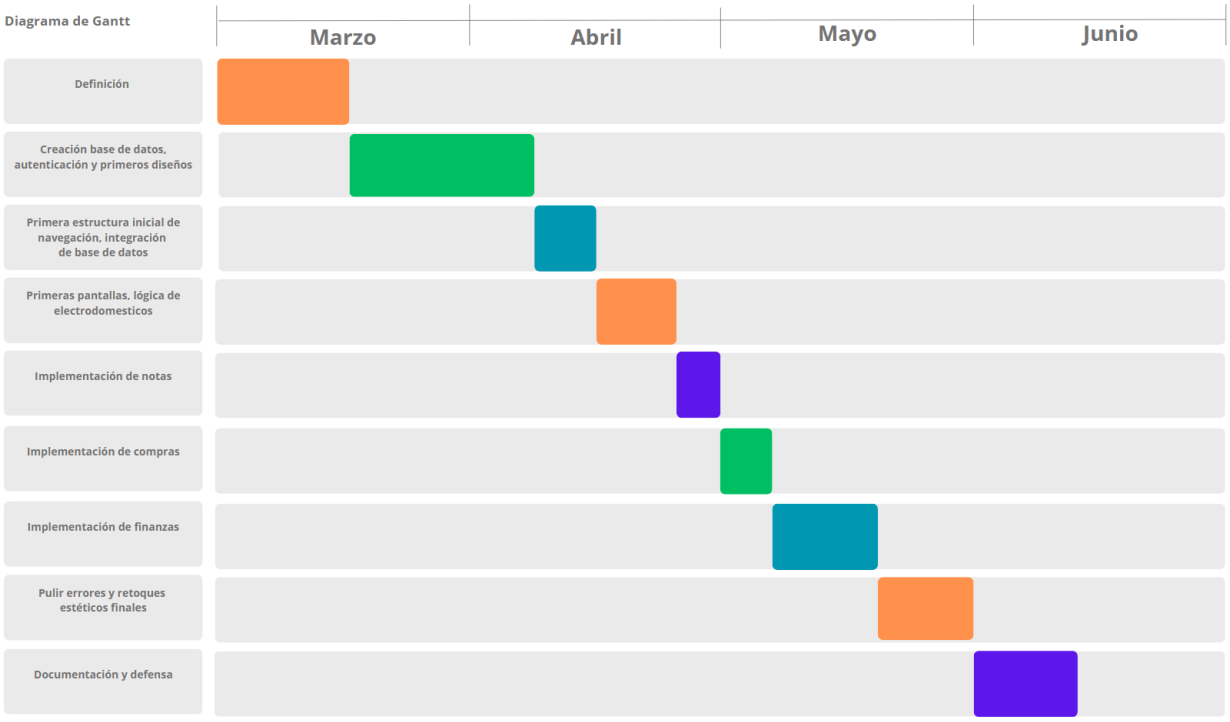
La planificación inicial ha sido la siguiente:

Fase0	Definición	10 mar 2025	16 mar 2025
Fase1	Creación base de datos, autenticación y primeros diseños	17 mar 2025	9 abr 2025
Fase2	Primera estructura inicial de navegación, integración de base de datos	10 abr 2025	15 abr 2025
Fase3	Primeras pantallas, lógica de electrodomésticos	16 abr 2025	24 abr 2025
Fase4	Implementación de notas	25 abr 2025	29 abr 2025
Fase5	Implementación de compras	29 abr 2025	05 may 2025

Fase6	Implementación de finanzas	06 may 2025	19 may 2025
Fase7	Pulir errores y retoques estéticos finales	20 may 2025	30 may 2025
Fase8	Documentación y defensa	31 may 2025	06 jun 2025

Para la planificación he usado la aplicación **Notion**, lo que me ha permitido llevar un correcto control del tiempo y las tareas.

2.2.2 Diagrama de Gantt



2.2.3 Comparativa entre planificación inicial y el desarrollo real

Durante el desarrollo se ha intentado seguir las fechas estipuladas, pero esto no siempre ha sido posible debido a diferentes problemas. Por ejemplo:

-Problemas relacionados con la lógica de electrodomésticos causaron un retraso de dicha implementación, lo que pospuso el inicio de los siguientes aspectos hasta que estos fueran solucionados.
-Durante todo el desarrollo ha tenido que haber rediseños, tanto en el aspecto visual como en la base de datos para mejorar la lógica a medida que se iba progrb vn besando.

-Factores externos al desarrollo también provocaron ciertos retrasos en algunos aspectos.

A pesar de esto, gracias a llevar un correcto control del tiempo y a la metodología empleada, la cual me ha permitido ser flexible, el desarrollo ha podido seguir con lo previsto dentro de las problemáticas experimentadas.

Respecto a estos problemas se desarrollará mas en los apartados posteriores implementación y Realización de pruebas

3. OBJETIVOS

Objetivo principal: crear una aplicación capaz de gestionar todas aquellas facetas compartidas de la vida en una casa por los habitantes de esta, haciendo mucho más sencilla su gestion y la convivencia.

Para ello como objetivos específicos tiene:

1. Crear / unirse a una “casa” y compartir toda la información de esta entre sus miembros.

2. Gestionar una lista de la compra colaborativa con categorías, productos recurrentes y borrado masivo de productos comprados.
3. Registrar, dividir y saldar gastos entre usuarios, mostrando deudas pendientes.
4. Añadir y controlar electrodomésticos (iniciar, pausar, reiniciar programas) con estado en vivo.
5. Crear y editar notas con autoguardado.

4. DESARROLLO DE LA APLICACIÓN

4.1 **Análisis de requisitos**

Requisitos que debe cumplir el sistema.

Gestión de casas:

La aplicación debe permitir creación de casas.
La aplicación debe permitir la sincronización de información de los usuarios de la casa
Al crear una casa se le asociará un código de invitación con el que se podrá invitar a otras personas.

Usuarios:

Capacidad de registrar, iniciar sesión y administrar usuarios.

Compras:

La aplicacion debe permitir crear, editar y borrar productos, así como marcarlos como recurrentes.

Los productos podrán marcarse como comprados.
Debe haber un botón que elimine todos los productos comprados
La lista debe mostrarse agrupada en categorias previamente creadas por los usuarios.
Gastos y finanzas:

Debe mostrarse un resumen de “Mis gastos”, “Gastos conjuntos” y “Deudas”
Podrá añadirse, editar y eliminar gastos y deudas.

Las deudas serán automáticamente calculadas en base a las cantidades y usuarios que hayan participado en dichos gastos.

Control de electrodomésticos:

Los usuarios deben poder iniciar, El sistema debe permitir registrar electrodomésticos, cada uno con sus programas (nombre, duración, consumo estimado) pausar y reiniciar el programa de un electrodoméstico y ver su estado en la pantalla principal.

Los usuarios deben poder iniciar, pausar y reiniciar el programa de un electrodoméstico y ver su estado en la pantalla principal.

Gestión de notas compartidas:

Las notas deben guardarse automáticamente al modificarse (en notas nuevas, el botón Guardar será obligatorio para la primera versión)

4.2 Requisitos no funcionales

La aplicación debe realizar las cargas en un tiempo no mayor a 2 segundos.
La aplicación debe validar correctamente las credenciales de usuario y no permitir la fuga de información.
La aplicación debe ser amigable e intuitiva.
La aplicación debe ser estéticamente moderna y llamativa

4.2. Diseño: arquitectura.

Cohabia ha sido desarrollado en **Kotlin** junto con **Jetpack Compose** para la UI debido a que este lenguaje es idóneo para el objetivo de esta aplicación móvil, la cual necesita recomposiciones en el UI de manera frecuente.

Diseño: Arquitectura
Patrón MVVM – 3 capas

1. **Model** → contiene la lógica de negocio y acceso a datos (Firebase Auth, Firestore, Storage).
2. **ViewModel** → gestiona el estado de la aplicación y la lógica de presentación, separando la UI de la lógica.
3. **View** → construida con Jetpack Compose, reacciona a los estados expuestos por el ViewModel.

Motivos de las tecnologías elegidas

- **MVVM:** estructura limpia que separa claramente responsabilidades, facilita el mantenimiento y la escalabilidad. Esta estructura es ideal para integrar la base de datos de Firebase con el resto del código.

- **Kotlin + Jetpack Compose:** lenguaje moderno con una UI declarativa que simplifica el desarrollo. Compose permite crear interfaces reactivas que se integran perfectamente con los estados observables del ViewModel

- **Koin:** permite una inyección de dependencias sencilla y eficiente, muy útil para desacoplar clases y facilitar el desarrollo.

- **Firebase (Auth, Firestore, Storage, FCM):** backend completo en la nube que permite eliminar la necesidad de un servidor propio. Ofrece autenticación de usuarios, base de datos en tiempo real, almacenamiento de archivos y sistema de notificaciones, todo bajo una misma plataforma.

- **Gradle** – Sistema de automatización de compilación para la gestión de dependencias y la construcción del proyecto. Elegido por ser la herramienta recomendada para el desarrollo en Android.

- **Lottie:** permite incluir animaciones con archivos json , ideales para representar estados de carga, éxito o error. Su facil integración y su completa biblioteca me han sido de mucha utilidad.

Integracion de estas tecnologias entre si:

- **Jetpack Compose** y **Firebase** se integran sin problemas. Además, gracias a que Firebase dispone de herramientas como “Authentication” o “Firebase Cloud Messaging”, la integración ha sido total.

- **Lottie** añade animaciones que mejoran mucho la experiencia del usuario, y se usa sin complicaciones dentro de Compose.

En resumen, todo funciona de forma fluida, está bien conectado y permite trabajar rápido sin complicarse.

4.2.2. Base de datos

Se ha elegido Firebase como base de datos debido a la estructura de la aplicación, ya que es una base de datos documental NoSQL basada en colecciones y documentos JSON.

Estructura documentos en Firebase:

Todos los ids son unicos y autogenerados por Firebase.

1. Colecciones principales:

-Casas:


```
{
  "miembros": [
    "x7OtpQTNCVVRMqyDTOvQkW1RUSq2",
    "Z3BxIn9rTPedGAe4AmH6mI5kCth1"
  ],
  "nombre": "Casa Ejemplo"
}
```


-miembros: en este array se guardan los ids de los usuarios pertenecientes a la casa
-nombre: nombre de la casa

-Usuarios:


```
{
  "casa": "Qr82m248zTfTrMQuZHJl",
  "correo": "miguelvelo@gmail.com",
  "fotoPerfil": "",
```

"**id**": "x7OtpQTNCVVRMqyDTOvQkW1RUSq2",
"**nombre**": "Miguel Velo"
}

-casa: referencia al id de la casa a la que pertenece dicho usuario, tanto si ha sido creada por él como si se ha unido a ella.
Id: id propio. Es generado por Firebase Authentication.

1.2 Subcolecciones

El resto de documentos al ser comunes a todos los integrantes de la casa deben ser subcolecciones de Casa.

-Gastos:
{
"**año**": 2025,
"**cantidad**": 100,
"**concepto**": "Compra comida",
"**dia**": 20,
"**fecha**": "2025-05-20T19:54:44+02:00",
"**hayDeuda**": true,
"**id**": "",
"**mes**": 5,
"**usuarioPaga**": [
"Z3BxIn9rTPedGAe4AmH6mI5kCth1"
],
"**usuariosDeuda**": [
"x7OtpQTNCVVRMqyDTOvQkW1RUSq2"
],
"**usuariosParticipan**": [
"Z3BxIn9rTPedGAe4AmH6mI5kCth1",
"x7OtpQTNCVVRMqyDTOvQkW1RUSq2"
]
}

Se guarda la fecha separada en día, mes y año para facilitar a la hora de mostrar gastos por ejemplo en el ultimo mes.
Los 3 arrays usuarioPaga, usuarioDeuda y usuariosParticipan guardan ids de usuarios que interaccionan con dicho gasto

En usuarioPaga se guardan aquellos que han pagado.
En usuariosParticipan aquellos que han participado en el gasto, es decir forman parte de el, hayan pagado o no.

En usuariosDeuda aquellos que participan y no han pagado aún.

Cuando un usuario en usuarioDeuda completa el pago pasa a estar en usuarioPaga

Ejemplos:
A , B y C han ido a cenar y pagan cada uno 25 euros. A ha pagado los 75 euros. B ya ha pagado su deuda y C aun no.
El gasto a registrar quedará:
usuarioPaga:A
usuariosParticipan:A,B,C
usuarioDeuda:C

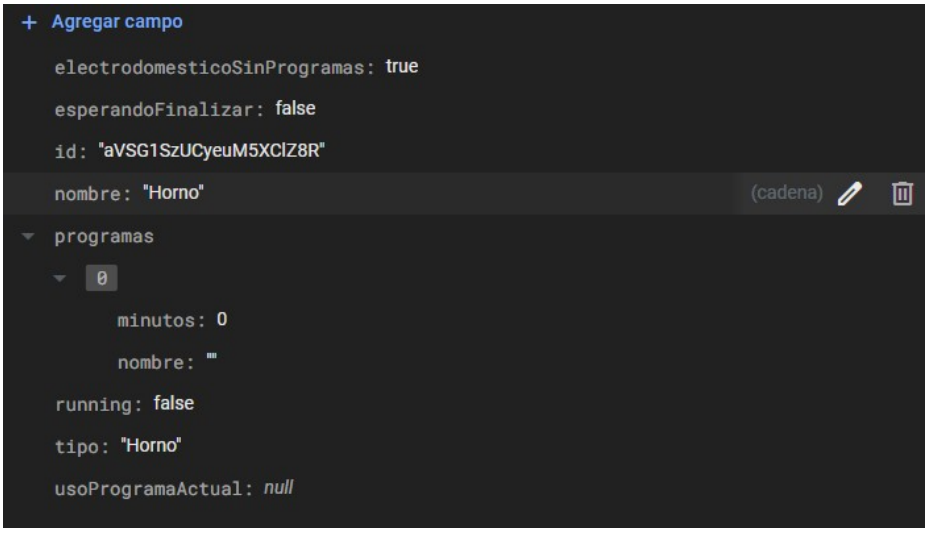
-Notas:

{
  "**contenido**": "Maleta, batería portátil y neceser",
  "**fecha**": "2025-05-20T20:21:33+02:00",
  "**id**": "",
  "**titulo**": "Lista cosas para el viaje"
}

-Electrodomésticos:

{
"**id**": "30C4l5TnscJu4Ax565L1",
"**nombre**": "ed",
"**programas**": [
    {
        "minutos": 90,
        "nombre": "sf",
        "running": false,
        "tipo": "Horno"
    }
],
"**usoProgramaActual**": null
}

Captura real de la base de datos del documento electrodoméstico cuando se encuentra sin funcionamiento:



-UsoPrograma (de los electrodomésticos):

{
"**electrodomesticoId**": "30C4l5TnscJu4Ax565L1",
"**estado**": "en_ejecucion",
"**id**": "",
"**inicio**": 1746102782207,
"**pausado**": false,
"**programaId**": "sf",
"**tiempoPausado**": 0,
"**timestampUltimaPausa**": null
}

4.3 Implementación

El desarrollo se ha llevado a cabo con las siguientes herramientas:

Android Studio
Usado como entorno de desarrollo ya que es el recomendado oficialmente para Kotlin.

Lenguajes de programación:
Kotlin ya que es el lenguaje oficial recomendado para una aplicación en Android

Frameworks
Jetpack Compose, elegido en lugar de XML por su modernidad y facilidad a la hora de crear interfaces. Debido a que su aprendizaje es más complejo de entrada ha sido necesario formarme en este ámbito en mayor profundidad antes de iniciar el proyecto.

Control de versiones:

Se ha llevado un control de las versiones creando ramas con cada implementación a un repositorio en Github.

4.3.1 Cambios y decisiones

A lo largo del desarrollo del proyecto han ido surgiendo ciertos problemas que han implicado soluciones sencillas en algunos casos y en otros cambios incluso estructurales. A continuación se relatan algunos de los cambios más importantes y la motivacion de estos, asi como caambios que son consecuencia de resolucion de errores.

-Posponer "Calendario" para trabajos futuros:

Tras estudiar los requerimientos para implementar un calendario se llego a la conclusión de que esto llevaria un tiempo excesivo lo que podría limitar otras funcionalidades más importantes, por eso tome la decision de posponer su implementacion.

-Cambios de diseño:

A medida que desarrollaba el proyecto iban surgiendo nuevas ideas de diseño y nuevas funcionalidades, lo cual implicaba reestructurar ciertas pantallas. Esto entraba dentro de lo previsible por lo que no ha necesitado grandes esfuerzos para solucionarse.

-Problemas por causas externas en el tiempo:

.

Por diversos motivos me vi obligado a reestructurar los tiempos, gracias a una buena organizacion y a tener todo planeado esto no supuso demasiado perjuicio, si bien es verdad que retrasó la implementacion de algunas funcionalidades.

-Problema con repositorio de GitHub:

Debido a un problema con el que estaba usando, tuve que cambiar el proyecto a otro repositorio nuevo. Gracias a tener varias copias tanto en local como en la nube este problema se solucionó fácilmente.

-Problemas en Modulo de Finanzas:

Un problema en el modulo de finanzas obligó a realizar diversas pruebas hasta hallar la solucion, ya que era un error critico. Tras varias pruebas encontre que el problema era causado por un formateo en los decimales que hacía que cada gasto tuviese distinto numero de digitos. Tras implementar la solucion todo funciona correctamente.

5 Realizacion de pruebas

5.1 Diseño de pruebas

Pruebas de caja blanca

Tipo de prueba	Prueba realizada	Resultado obtenido
Caja blanca	Verificación del flujo de lógica en el login	Acceso exitoso con credenciales válidas. Error controlado con inválidas.
Caja negra	Registro de usuario con email duplicado	Firebase devuelve error y se muestra mensaje adecuado.
Caja blanca	Validación del formato de todos los textfields (email, contraseña, nombre, etc.)	Algunos han reportado errores por lo que se ha procedido al correcto control de estos
Caja blanca	Revisión de botones y que permitan avanzar solo cuando es debido	Todos los botones respetan las condiciones y solo permiten avanzar cuando corresponde.
Caja negra	Testeo del correcto funcionamiento de pantalla inicial (dicha pantalla redirige en base a si se ha verificado correo, si hay sesion iniciada y si hay casa creada)	Verificado el correcto funcionamiento, se han probado todas las posibilidades, asi como probar a cortar el proceso de creacion de golpe. Redirige correctamente en cualquier situación.
Caja negra	Testeo de generación de deudas a partir de la creación de nuevos gastos	Detectados algunos fallos en el formato de los decimales. Posteriormente resuelto.
Caja negra	Gestión del estado de los electrodomésticos (encender, apagar, contar uso)	Funcionamiento verificado. Se actualiza el estado correctamente y se guarda en Firestore.

Los principales errores detectados fueron:

Fallo en la validación de campos de entrada: Algunos textfields permitían avanzar con datos inválidos. Este problema fue corregido implementando controles de validación estrictos.

Error en el formato decimal: En la generación automática de deudas, algunos cálculos se realizaban incorrectamente debido al numero de decimales distinto en algunos apartados. Tras posteriores ajustes todo funciona correctamente.

La conclusión tras las pruebas es que el sistema es estable y fiable en condiciones normales. Si bien es posible que posteriormente se pueda encontrar algún error, este no sera crítico.

Se han ejecutado pruebas muy variadas de distinto ámbitos por lo que el resultado final es satisfactorio.

6.Conclusiones

Realizar este trabajo me ha sido muy útil a la hora de mejorar mis habilidades como programador.
Debido a la dificultad y extensión de este me he visto en la necesidad de ampliar mucho mis conocimientos en ciertas tecnologías o lenguajes como Kotlin o Firebase, realizando cursos, leyendo documentación y leyendo mi propio código en reiteradas ocasiones.

He reafirmado que el desarrollo de software es realmente lo que me apasiona. Aunque ya lo intuía, enfrentarme a un reto tan complejo como este me ha confirmado que quiero seguir avanzando en este camino profesional, incluso con la posibilidad de terminar lanzando esta aplicación al público.

A pesar de que ha habido varios obstáculos a lo largo del desarrollo creo que he sabido solventarlos de una manera bastante eficiente lo cual considero que me prepara muy bien para el futuro laboral en este sector, donde este tipo de problemáticas son continuas y la necesidad de adaptación es clave.

En resumen estoy muy contento con el resultado final de dicho proyecto y con los conocimientos adquiridos, sintiéndome un paso más adelante en mi carrera como programador respecto al inicio de este proyecto.

7. Trabajo futuro

Debido a que me gustaria seguir con el progreso de dicha aplicacion tengo una amplia lista de posibilidades para trabajo futuro.

General:

-Implementación de autenticacion con más posibilidades como , por ejemplo, Google.

-Desarrollo de generación de QR para invitacion a casa
-Integracion de las APIs de diferentes marcas de electrodomesticos para recibir datos auténticos en tiempo real.
-Documentacion legal al respecto
-Añadir doble factor de autenticación para mejorar la seguridad de acceso.
-Implementación de un backend en Node.js o Spring Boot para funciones que requieran lógica compleja o integración externa.
-Cambio en el almacenamiento de fotos y archivos (como fotos, iconos de electrodomésticos o logo) de local a base de datos o backend.

Interfaz:

-Implementación de modo oscuro.
-Implementación de Widgets en la pantalla principal de dispositivo.
-Posibilidad de cambiar foto de perfil del usuario.

Notas:

-Implementar creacion de carpetas de notas.

Compras y gastos:

-Estudio de viabilidad a la hora de implementar ML KIT, una herramienta de Machine Learning, la cual entre otras funciones permitira el reconocimiento de texto a traves de la camara. De esta manera y en conjunto con la API de Gemini , se implementaria la funcionalidad de reconocimiento de tickets automático, tachando de Compras los productos adquiridos automaticamente y añadiendo en Gastos el precio reflejado en dicho ticket.

-Análisis de gastos mensuales mediante gráficos y categorización automática.

Expansion de nuevos Modulos:

-Creacion de calendario como estaba previsto inicialmente.
-Módulo de documentos en el que se pueden guardar documentos de interés común como facturas, tickets, seguros y más.

8. Bibliografía

JetBrains. (n.d.). Kotlin documentation. Kotlin. https://kotlinlang.org/docs/home.html
Google. (n.d.). Firebase documentation. Firebase. https://firebase.google.com/docs

Airbnb. (n.d.). Lottie for Android. GitHub. https://github.com/airbnb/lottie-android

Mathieu, R. (2023, July 11). Discovering Material 3 for Android: ListItem. Medium. https://medium.com/@renaud.mathieu/discovering-material-3-for-android-listitem-62f30b3cad68

user1166906. (2019, May 24). How do I split the date from the string of date time with time zone? Stack Overflow. https://stackoverflow.com/questions/56289530/how-do-i-split-the-date-from-the-string-of-date-time-with-time-zone

Google. (s. f.). *Firebase Authentication*. Firebase. https://firebase.google.com/docs/auth?hl=es-419

Google. (s. f.). *Documentación de Jetpack Compose*. Android Developers. https://developer.android.com/develop/ui/compose/documentation?hl=es-419

MoureDev by Brais Moure. (2022, 3 de enero). *Cómo estudiar programación* [Video]. YouTube. https://www.youtube.com/watch?v=yVIGAvMO3bc